LFTC: Lab2 - use of GenAI

Link to the chat (*chatGPT)* : https://chatgpt.com/share/6acb9b83-0ca0-83ed-bb22-41f954f70c6e

Me:

Validate the following BNF grammar for a mini-language designed for data quality checks on integer datasets.
Check for:

- Undefined nonterminals, missing rules or rules that cannot be reached from `<program>`.
- Ambiguities and whether arithmetic precedence and associativity are correctly represented.
- Inconsistencies or unintended restrictions in the syntax.

For each issue, explain the problem and suggest a minimal correction. Distinguish actual errors from optional improvements. Do not treat semantic checks, such as variable initialization or array bounds, as BNF errors.
Notation: `<...>` marks nonterminals, unquoted symbols are terminals, `|` separates alternatives, and `ε` represents the empty string. Headings are only for readability. The start symbol is `<program>`.
BNF grammar: *[BNF text file]*

GenAI:

The grammar is structurally valid. There are no undefined nonterminals, unreachable rules, or missing productions needed to derive a program. Arithmetic precedence and associativity are correctly represented. The main

points to clarify concern lexical conventions and intentional syntax restrictions.

1. Nonterminals and reachability — no errors

Every nonterminal used on a right-hand side has a definition, and every rule is reachable from `<program>`.
All nonterminals can derive finite terminal strings. Recursive rules such as `<expression>`, `<factor>`, and `<identifier-tail>` have terminating alternatives.

2. Arithmetic precedence and associativity — correct

The grammar establishes these precedence levels, from highest to lowest:

Operators	Representation	Behaviour
Unary `-`	`<factor> ::= - <factor>`	Binds more tightly than multiplication
`*`, `/`, `%`	`<term>`	Equal precedence, left-associative
`+`, `-`	`<expression>`	Equal precedence, left-associative

For example:

- `a + b * c` parses as `a + (b * c)`.
- `a - b - c` parses as `(a - b) - c`.
- `a / b % c` parses as `(a / b) % c`.
- `a * b` parses as `(-a) * b`.

Parentheses explicitly override precedence. **No correction is necessary.**
The left recursion is valid BNF. It would require transformation for some parsing approaches, such as a basic recursive-descent parser, but that is a parser implementation concern.

3. Conditional statements — no dangling- `else` ambiguity

Both branches require a `<block>`, and every block is enclosed in braces. Consequently, an `else` cannot ambiguously attach to different nested `if` statements.
No correction is necessary.

4. Keywords overlap with identifiers — lexical clarification

The `<identifier>` rule can generate words such as `int`, `if`, `in`, and `unique`. Therefore, the grammar alone does not prohibit examples such as:

```
int if;
if = 1;
```

This is **not automatically a grammatical ambiguity or an error**: allowing keywords as identifiers can be a deliberate language choice. However, a scanner that always classifies these words as keywords would reject some strings permitted by the identifier production.
Minimal correction if keywords should be reserved: add a lexical convention:

> Reserved keywords cannot be used as identifiers. An identifier matching a reserved keyword is classified as that keyword.

No arithmetic or statement productions need to change.

5. Tokenization and whitespace — lexical clarification

The specification does not explicitly define whitespace handling or token boundaries. It also writes `read(` and `print(` together, while other punctuation is separated.
That does not make the BNF invalid, but it leaves unclear whether `read(` is one terminal or the two terminals `read` and `(`, and whether `read (x);` is accepted.
Minimal clarification:

> Whitespace is ignored between tokens. Keywords, identifiers, integer constants, operators, and punctuation are tokens. Tokenization uses longest match.

For consistency, write:

```
<input> ::= read ( <reference> ) ;
<output> ::= print ( <expression> ) ;
```

Longest match also clarifies the handling of operators such as `<` versus `<=`, and `=` versus `==`.

6. Syntax restrictions — valid, but worth confirming

These are **optional improvements**, not actual BNF errors.

Current restriction	Consequence	Minimal change, only if desired
`<declarations>` is nonempty	Every program must contain at least one declaration	Add `ε` as an alternative
`<statements>` is nonempty	Every program and block must contain a statement	Permit an empty statement sequence
Declarations occur only at the start of `<program>`	No declarations inside blocks or after statements	Add `<declaration>` to `<statement>` if declarations should be allowed there
Array sizes use `<positive-integer>`	Sizes must be positive integer literals; `int a[n];` is rejected syntactically	Use `<expression>` if computed sizes are intended
Conditions have no Boolean combinations	No `and`, `or`, or general Boolean `not`	Add Boolean-expression rules with explicit precedence
Membership and uniqueness use `<identifier>`	Their dataset operand must be a named identifier	Broaden the operand only if other dataset forms are supported

To allow empty blocks while keeping the requirement for at least one top-level statement:

```
<block> ::= { <optional-statements> }
<optional-statements> ::= <statements> | ε
```

Other valid design choices include rejecting leading-zero literals such as `007`, identifiers beginning with `_`, unary `+`, and declaration initializers such as `int x = 5;`. None requires correction unless the language is intended to support it.

7. Semantic checks — outside BNF validation

The following are semantic concerns and should **not** be reported as grammar errors:

- Whether variables are declared or initialized before use.
- Whether a reference denotes a scalar or an array.
- Whether the operand of `in` or `is unique` is an integer dataset.
- Whether an array index is within bounds.
- Whether division uses a nonzero divisor.
- Whether the endpoints of `between` form a meaningful interval.

Verdict: no definite structural or arithmetic errors were found. Clarify keyword handling and tokenization, then retain or relax the listed restrictions according to your intended language.

Me:

Based on your review, I plan to update the grammar manually to support:

1. Programs without declarations.
2. Empty blocks and programs without statements, including an empty program.
3. Arithmetic expressions as array sizes.
4. General negation using `not (<condition>)`, replacing the separate `not in` alternative.
5. Optional initialization of scalar variables at declaration.

Would these changes introduce any ambiguity or conflict with the existing grammar? Identify any points I should consider, but do not rewrite the productions or provide an updated grammar.

GenAI:

These changes can fit the existing grammar without introducing ambiguity, provided empty sequences and negation are handled consistently.

1. **Programs without declarations.** Making the declaration sequence optional is safe. Declarations begin with `int`, while statements have different forms. Reserving `int` as a keyword keeps this distinction clear.
2. **Empty blocks and programs.** Allowing both declaration and statement sequences to be empty permits an empty program. Avoid giving the same sequence multiple ways to derive emptiness—for example, combining an optional wrapper with an already nullable sequence. Also avoid making an individual `<statement>` empty: repetitions of empty statements could introduce ambiguity. Make the **sequence** optional instead.
3. **Arithmetic expressions as array sizes.** This creates no conflict: `[` and `]` delimit the size expression, and the existing expression rules already establish precedence. Expressions such as `n + 1`, `0`, or `3` would become syntactically valid sizes. Whether their values are positive and when they are evaluated are semantic decisions.
4. **General negation with `not (<condition>)`.** The keyword and mandatory parentheses make the scope explicit, so this should be unambiguous. It also supports nested negation. After removing `not in`, membership negation would be written as `not (x in dataset)`; `x not in dataset` would no longer be accepted. Reserve `not` as a keyword.
5. **Optional scalar initialization.** This is unambiguous because the token after the scalar identifier distinguishes an ordinary declaration (`;`), an initialized declaration (`=`), and an array declaration (`[`). Keep initialization restricted to the scalar form if that is your intention. Whether an initializer references an undeclared variable or the variable being declared is a semantic concern.

The main structural point is **one unambiguous way to represent an empty sequence**. The changes do not affect arithmetic precedence, associativity, or the existing protection against dangling-`else` ambiguity.